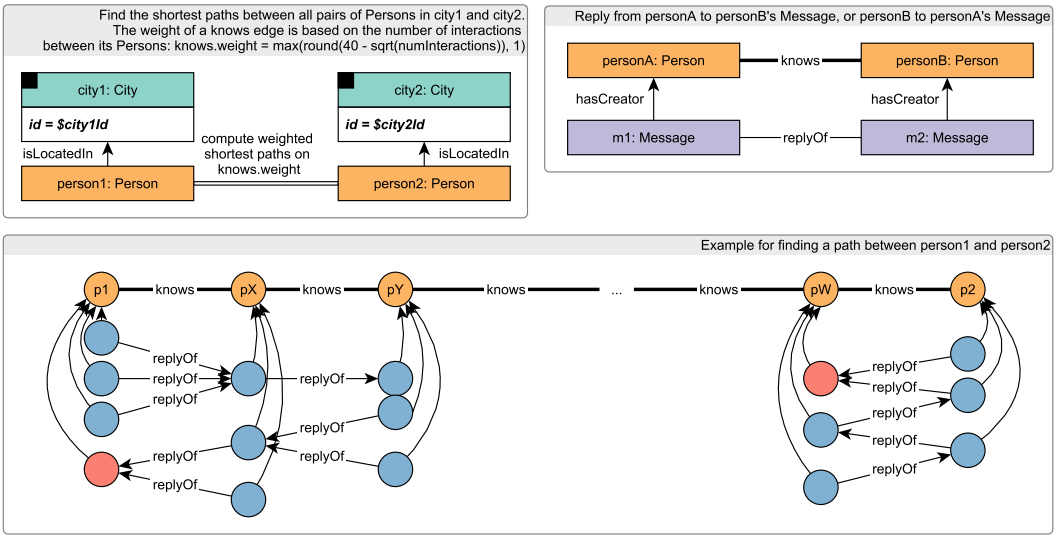
BI / read / 19

BI 1
BI 2
BI 3
BI 4
BI 5
BI 6
BI 7
BI 8
BI 9
BI 10
BI 11
BI 12
BI 13
BI 14
BI 15
BI 16
BI 17
BI 18
BI 19
BI 20

query	BI / read / 19
title	Interaction path between cities
pattern	
description	Given two Cities with IDs `$city1Id`, `$city2Id`, find Persons `person1`, `person2` living in these Cities (respectively) with the *cheapest* interaction path between them. The cheapest path is equivalent to the *weighted shortest* path. It is computed on a subgraph of the Person-knows-Person graph with the edge weights based on the number of interactions. An *interaction* is a direct reply Comments from one Person to Messages by the other Person. Only knows edges with at least one interaction between their endpoint Persons are considered. For these, the weight of a knows edge is defined as: $\max(\mathrm{round}(40 - \sqrt{numInteractions}), 1)$ If there are multiple pairs of people with cheapest paths that have the same total weight, return all of them. *Note:* Interactions are counted both ways, e.g. if Alice knows Bob, Alice writes 2 reply Comments to Bob's Messages and Bob writes 3 reply Comments to Alice's Messages, their total number of interactions is 5 and the weight of the knows edge is 38. *Remark:* Determinism is ensured by using square root followed by rounding. For all integers between 1 and 100 000, the square root's fractional part is more than 10e-5 from 0.5, where the rounding could be non-determinstic based on floating point inaccuracies. As 10e-5 is significantly larger than the machine epsilon of IEEE 754 floats (both 32- and 64-bit), the floating point inaccuracies have no chance to affect the derived integer edge weights.

params

1	`$city1Id`	ID	(a) Small Cities within the same Country (b) Larger Cities from different Countries
2	`$city2Id`	ID	

result

1	`person1.id`	ID	R	
2	`person2.id`	ID	R	
3	`totalWeight`	32-bit Integer	C	

sort

1	`person1.id`	↑	
2	`person2.id`	↑	

limit	n/a
CPs	3.3, 7.6, 7.7, 8.4, 8.6
relevance	To find the weighted shortest paths efficiently, the system can use e.g. a bidirectional Dijkstra algorithm. As the edge weights do not depend on any parameter, systems can pre-compute them (if they do not interleave reads and writes).